



A COMPILER CHECKS WHETHER PARENTHESES IN A PROGRAM ARE BALANCED DURING SYNTAX ANALYSIS

**UNDER THE GUIDANCE OF:
DR.J.ALPHONSA**

presented by :
A Velmurugan(192521074)



INTRODUCTION

- A compiler translates source code into machine code.
 - Syntax analysis is the second phase of a compiler.
 - It checks whether the program follows grammar rules.
 - Balanced parentheses are essential for correct syntax.
 - Unbalanced parentheses produce syntax errors.
 - This check is performed before code generation
- 



PROBLEM STATEMENT

- The compiler must verify matching opening and closing brackets.
 - Every (should have a corresponding).
 - Brackets must be properly nested.
 - Incorrect parentheses lead to compilation errors.
 - Example: $(a+b)*(c-d)$ is balanced, while $(a+b*(c-d)$ is not.
- 



DATA STRUCTURE – STACK

- Stack is the most suitable data structure.
 - It follows the **LIFO (Last In, First Out)** principle.
 - It stores opening parentheses. It removes matching parentheses in reverse order.
 - It efficiently handles nested expressions.
 - It is widely used in compiler design
- 

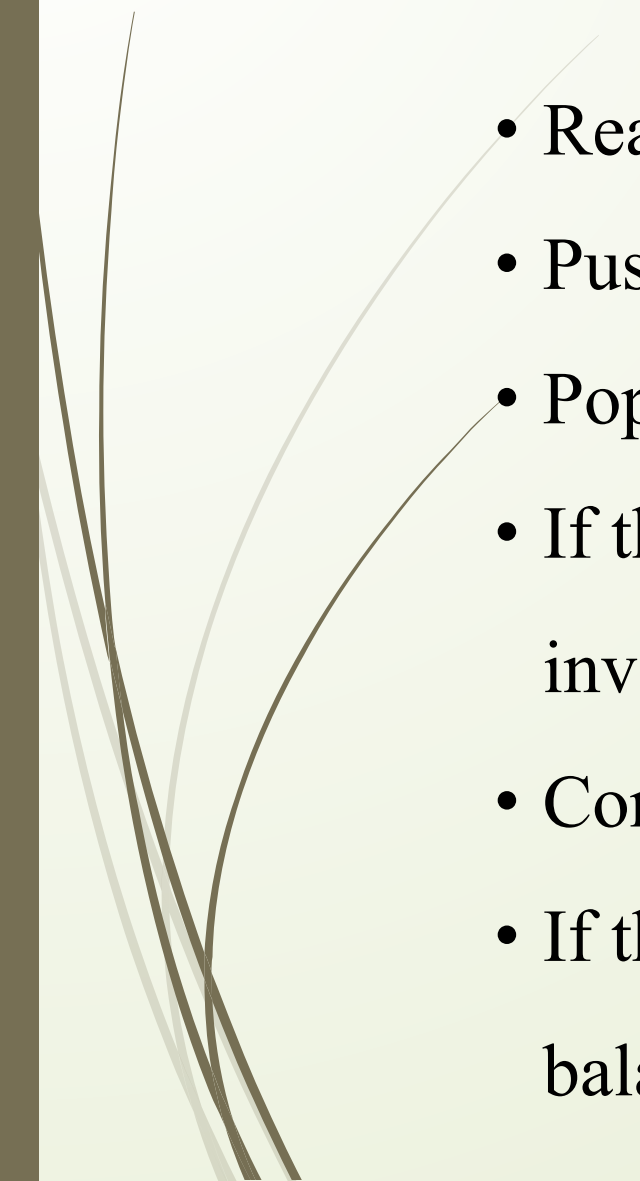


STACK OPERATIONS

- **Push:** Adds an opening parenthesis (to the stack.
- **Pop:** Removes the top opening parenthesis when) is found.
- **Peek:** Checks the top element without removing it.
- **IsEmpty:** Checks whether the stack is empty.
- These operations take constant time **$O(1)$** .
- They make syntax checking fast and efficient.



WORKING PROCESS

- Read the expression from left to right.
 - Push every opening parenthesis onto the stack.
 - Pop the top element when a closing parenthesis appears.
 - If the stack is empty before popping, the expression is invalid.
 - Continue until all characters are processed.
 - If the stack is empty at the end, the parentheses are balanced.
- 



TIME COMPLEXITY

- Each parenthesis is processed only once.
- Push operation takes **$O(1)$** time.
- Pop operation takes **$O(1)$** time.
- Overall algorithm runs in **$O(n)$** time.
- Space complexity is **$O(n)$** in the worst case.
- Suitable for large programs.



CONCLUSION

- Stack is the ideal data structure for checking balanced parentheses.
 - Its LIFO property perfectly matches nested brackets.
 - It quickly detects syntax errors.
 - The algorithm is simple and efficient.
 - Time complexity is **$O(n)$** with reliable performance.
- 